# Паттерн State

1. **Детерминированный конечный автомат:**

$\boldsymbol{A} = \langle \boldsymbol{S}, \boldsymbol{I}, \boldsymbol{\delta}, \boldsymbol{s_0}, \boldsymbol{F} \rangle$ **–** конечный детерминированный автомат;

$\boldsymbol{S}$ – множество состояний;

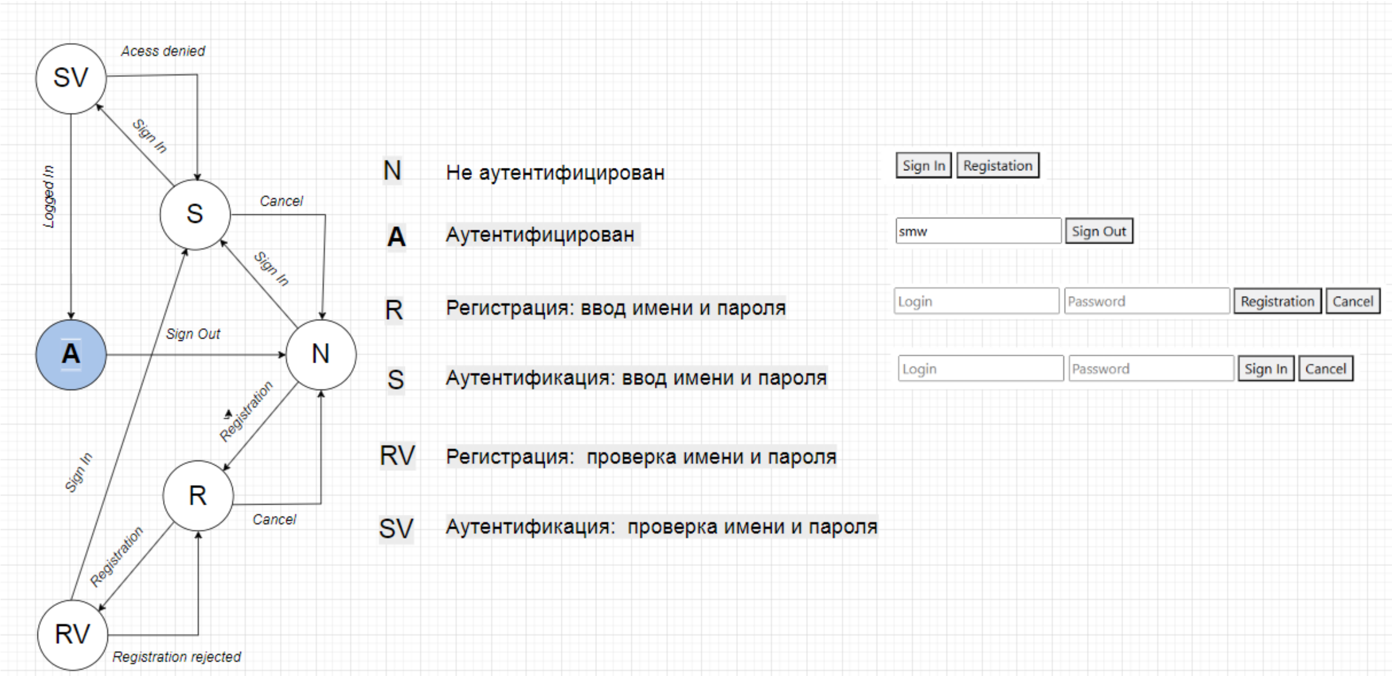
$\boldsymbol{I}$ – алфавит;

$\boldsymbol{\delta}: \boldsymbol{I} \times \boldsymbol{S} \rightarrow \boldsymbol{S}$, $|\boldsymbol{\delta}(\boldsymbol{s}, \boldsymbol{i})| \leq \mathbf{1}$, $\boldsymbol{s} \in \boldsymbol{S}, \boldsymbol{i} \in \boldsymbol{I}$ – функция переходов;

$\boldsymbol{s_0} \in \boldsymbol{S}$ – начальное состояние;

$\boldsymbol{F} \subset \boldsymbol{S}$ – конечные (допускающие) состояния.

2. **Алгоритм в форме конечного автомата:**




3. **Реализация: JS**

```
const API_REGISTRATION = '/api/Sec/Registration';
const API_SIGNIN       = '/api/Sec/SignIn';
const API_SIGNOUT      = '/api/Sec/SignOut';
const API_GETUSERNAME  = '/api/Sec/GetUserName';
```

```
class SA {      // finite state machine
    constructor(startState) {
        this.state = (SA.START[startState] != undefined) ? startState : SA.START.A; // state
        this.subscribers = {};                                                      // subscribers list
        this.on = (state, callback) => {                                            // subscribe
            if (!this.subscribers[state]) {
                this.subscribers[state] = [];
            }
            this.subscribers[state].push(callback);
        };
        this.emit = (state, data) => {                                              // notify
            if (this.subscribers[state]) this.subscribers[state].forEach(callback => callback(data));
        };
        this.RunCommand = (command) => {
            let s = SA.TRANFUNC[this.state][command];                               // execute command
            console.log(this.state, ':RunCommand: command =', command, '--->', s);
            if (s != undefined) {
                this.state = s;
                console.log('RunCommand: new state =', this.state);
                this.emit(this.state, SA.TRANFUNC[this.state]);
            }
            return { [this.state]: SA.TRANFUNC[this.state] };
        };
    };
```

```
    static START = {   // start states
        A: 'A',         // -- authorized
        N: 'N',         // -- authorized
    };
    static COMMAND = { // command
        REG: 'REG',     // -- registration
        SIN: 'SIN',     // -- sign in
        CAN: 'CAN',     // -- cancel
        ADE: 'ADE',     // -- access denird
        LIN: 'LIN',     // -- logged in
        SOU: 'SOU',     // -- sign out
        REJ: 'REJ'      // -- registration rejected
    };
    static STATES = {   // states
        A: SA.START.A,  //  -- authorized
        N: SA.START.N,  //  -- authorized
        S: 'S',         //  -- get sign in
        R: 'R',         //  -- get registration
        SV: 'SV',       //  -- validation sign in
        RV: 'RV'        //  --validation reistration
    };
    static TRANFUNC = {  // transition function
        [SA.STATES.A]: { [SA.COMMAND.SOU]: SA.STATES.N },
        [SA.STATES.N]: { [SA.COMMAND.SIN]: SA.STATES.S,   [SA.COMMAND.REG]: SA.STATES.R },
        [SA.STATES.S]: { [SA.COMMAND.SIN]: SA.STATES.SV, [SA.COMMAND.CAN]: SA.STATES.N },
        [SA.STATES.R]: { [SA.COMMAND.REG]: SA.STATES.RV, [SA.COMMAND.CAN]: SA.STATES.N },
        [SA.STATES.SV]: { [SA.COMMAND.LIN]: SA.STATES.A, [SA.COMMAND.ADE]: SA.STATES.S },
        [SA.STATES.RV]: { [SA.COMMAND.SIN]: SA.STATES.S, [SA.COMMAND.REJ]: SA.STATES.R }
    };
};
```

```
async function GetUserName(setA, setN) {
    let res = await fetch(API_GETUSERNAME);
    if ((await res.status) == 200) {
        let np = await res.json();
        console.log('GetUserName =', np);
        setA(np.name);
    }
    else { sa.RunCommand(SA.COMMAND.SOU); setN(); }
};
```

```
async function Registration(name, pass, sin, rej) {
    let res = await fetch(API_REGISTRATION, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json;charset=utf-8' },
        body: JSON.stringify({ name: name, pass: pass })
    });
    let rc = false;
    if (rc = ((await res.status) == 200)) { sa.RunCommand(SA.COMMAND.SIN); sin(); }
    else { sa.RunCommand(SA.COMMAND.REJ); rej(); }
    return rc;
};
async function SignIn(name, pass, lin, ade) {
    let rc = false;
    let res = await fetch(API_SIGNIN, {
        method: 'POST',
        credentials: 'include',
        headers: { 'Content-Type': 'application/json;charset=utf-8' },
        body: JSON.stringify({ name: name, pass: pass })
    });
    if (rc = ((await res.status) == 200)) { sa.RunCommand(SA.COMMAND.LIN); lin(); }
    else { sa.RunCommand(SA.COMMAND.ADE); ade(); }
    return rc;
};
async function SignOut(sou) {
    let rc = false;
    let res = await fetch(API_SIGNOUT, {
        method: 'POST',
        credentials: 'include'
    });
    if (rc = ((await res.status) == 200)) { sa.RunCommand(SA.COMMAND.SOU); sou(); }
    return rc;
};
```

```
<div class="col-5 d-flex justify-content-end ">
    <div id="STATE_N" style="margin-right:2%; display:none;">
        <button id="NSignIn" type="button"> Sign In</button>
        <button id="NRegistration" type="button">Registation</button>
    </div>
    <div id="STATE_A" style="margin-right: 2%; display:none;">
        <input type="text" id="AName" readonly />
        <button id="ASignOut" type="button">Sign Out</button>
    </div>
    <div id="STATE_R" style="margin-right: 2%; display:none;">
        <input id="RName" name="RName" type="text" placeholder="Login" />
        <input id="RPass" name="RPass" type="password" placeholder="Password" />
        <button id="RRegistration" type="button">Registration</button>
        <button id="RCancel" type="button">Cancel</button>
    </div>
    <div id="STATE_S" style="margin-right: 2%; display:none;">
        <input id="SName" name="NName" type="text" placeholder="Login" />
        <input id="SPass" name="NPass" type="password" placeholder="Password" />
        <button id="SSignIn" type="button">Sign In</button>
        <button id="SCancel" type="button">Cancel</button>
    </div>
</div>
```

```
let Visio = (state, name) => {
    if (state == SA.STATES.A) {
        if (name === undefined) { SName.value = ''; }
        else { AName.value = name; }
        STATE_N.style.display = STATE_R.style.display = STATE_S.style.display = 'none';
        STATE_A.style.display = 'block';
    } else
    if (state == SA.STATES.N) {
            STATE_A.style.display = STATE_R.style.display = STATE_S.style.display = 'none';
            STATE_N.style.display = 'block';
    } else
    if (state == SA.STATES.R) {
        RName.value = RPass.value = '';
        STATE_A.style.display = STATE_N.style.display = STATE_S.style.display = 'none';
        STATE_R.style.display = 'block';
    } else
    if (state == SA.STATES.S) {
        SPass.value = '';
        if (name === undefined) { SName.value = ''; }
        else { SName.value = name; }
        STATE_A.style.display = STATE_N.style.display = STATE_R.style.display = 'none';
        STATE_S.style.display = 'block';
    }
};
```

```
let sa = new SA(SA.START.A);
GetUserName(name => Visio(sa.state, name), () => Visio(sa.state));
```

```
NRegistration.onclick = () => {
    console.log('NRegistration = ', sa.RunCommand(SA.COMMAND.REG));
    Visio(sa.state);
};
NSignIn.onclick = () => {
    console.log('NSignIn = ', sa.RunCommand(SA.COMMAND.SIN));
    Visio(sa.state);
};
SSignIn.onclick = () => {
    console.log('SSignIn = ', sa.RunCommand(SA.COMMAND.SIN));
    Visio(sa.state);
};
SCancel.onclick = () => {
    console.log('SCancel = ', sa.RunCommand(SA.COMMAND.CAN));
    Visio(sa.state);
};
RRegistration.onclick = () => {
    console.log('RRegistration = ', sa.RunCommand(SA.COMMAND.REG));
    Visio(sa.state);
};
RCancel.onclick = () => {
    console.log('RCancel = ', sa.RunCommand(SA.COMMAND.CAN));
    Visio(sa.state);
};
ASignOut.onclick = () => {
    SignOut(() => Visio(sa.state));
};
```

```
sa.on(SA.STATES.RV, (state) => {
    console.log('on  RV  = ', state);
    Registration(RName.value, RPass.value,
        () => Visio(sa.state, RName.value),
        () => Visio(sa.state)
    );
});
sa.on(SA.STATES.SV, (state) => {
    SignIn(SName.value, SPass.value, () => Visio(sa.state, SName.value), () => Visio(sa.state));
});
```

4. **Реализация JS/Module**

```
export default class API {
    static  REGISTRATION =  'http://nginx.belstu.by/api/Sec/Registration';
    static  SIGNIN       =  'http://nginx.belstu.by/api/Sec/SignIn';
    static  SIGNOUT      =  'http://nginx.belstu.by/api/Sec/SignOut';
    static  GETUSERNAME  =  'http://nginx.belstu.by/api/Sec/GetUserName';
};
```

```
export class SAID {
    constructor(ifless, ifequal) {
        this.channel  = new BroadcastChannel('SAID');
        this.getSAID  = () => sessionStorage.SAID ? sessionStorage.SAID : (sessionStorage.SAID = (new Date()).getTime());
        this.postSAID = () => this.channel.postMessage({ "SAID": this.getSAID() });
        this.close    = () => this.channel.close();
        this.ifless   = ifless || (() => true);
        this.ifequal  = ifequal|| (() => true);
        this.channel.onmessage = (event) => {
            if (event.data.SAID < this.getSAID())       this.ifless();
            else if (event.data.SAID == this.getSAID()) this.ifequal()
            else this.postSAID();
        };
    };
};
```

```
export class SA {        // finite state machine
    constructor(startState) {
        this.state = (SA.START[startState] != undefined) ? startState : SA.START.A; // state
        this.subscribers = {};                                                     // subscribers list
        this.on = (state, callback) => {                                           // subscribe
            if (!this.subscribers[state]) { this.subscribers[state] = [];}
            this.subscribers[state].push(callback);
        };
        this.emit = (state, data) => {                                             // notify
            if (this.subscribers[state]) this.subscribers[state].forEach(callback => callback(data));
        };
        this.RunCommand = (command) => {
            let s = SA.TRANFUNC[this.state][command];                              // execute command
            console.log(this.state, ':RunCommand: command =', command, '--->', s);
            if (s != undefined) {
                this.state = s;
                console.log('RunCommand: new state =', this.state);
                this.emit(this.state, SA.TRANFUNC[this.state]);
            }
            return { [this.state]: SA.TRANFUNC[this.state] };
        };
    };
```

```
    static START = {    // start states
        A: 'A',          // -- authorized
        N: 'N',          // -- not authorized
    };
    static COMMAND = { // command
        REG: 'REG',     // -- registration
        SIN: 'SIN',     // -- sign in
        CAN: 'CAN',     // -- cancel
        ADE: 'ADE',     // -- access denied
        LIN: 'LIN',     // -- logged in
        SOU: 'SOU',     // -- sign out
        REJ: 'REJ'      // -- registration rejected
    };
    static STATES = {   // states
        A: SA.START.A,  //  -- authorized
        N: SA.START.N,  //  -- not authorized
        S: 'S',         //  -- get sign in
        R: 'R',         //  -- get registration
        SV: 'SV',       //  -- validation sign in
        RV: 'RV'        //  -- validation reistration
    };
    static TRANFUNC = {  // transition function
        [SA.STATES.A]:  { [SA.COMMAND.SOU]: SA.STATES.N },
        [SA.STATES.N]:  { [SA.COMMAND.SIN]: SA.STATES.S, [SA.COMMAND.REG]: SA.STATES.R },
        [SA.STATES.S]:  { [SA.COMMAND.SIN]: SA.STATES.SV,[SA.COMMAND.CAN]: SA.STATES.N },
        [SA.STATES.R]:  { [SA.COMMAND.REG]: SA.STATES.RV,[SA.COMMAND.CAN]: SA.STATES.N },
        [SA.STATES.SV]: { [SA.COMMAND.LIN]: SA.STATES.A, [SA.COMMAND.ADE]: SA.STATES.S },
        [SA.STATES.RV]: { [SA.COMMAND.SIN]: SA.STATES.S, [SA.COMMAND.REJ]: SA.STATES.R }
    };
};
```

```
export async function Registration(name, pass, sin, rej) {
    let res = await fetch(API.REGISTRATION,{ method: 'POST', headers: { 'Content-Type': 'application/json;charset=utf-8' },
                                        body: JSON.stringify({ name: name, pass: pass })
    });
    let rc = false;
    if (rc = ((await res.status) == 200))  sin();  else  rej();
    return rc;
};
export async function SignIn(name, pass, lin, ade) {
    let rc = false;
    let res = await fetch(API.SIGNIN, {method: 'POST', credentials: 'include',
                                    headers: { 'Content-Type': 'application/json;charset=utf-8' },
                                    body: JSON.stringify({ name: name, pass: pass })
    });
    if (rc = ((await res.status) == 200)) lin(); else  ade();
    return rc;
};
export async function SignOut(sou) {
    let rc = false;
    let res = await fetch(API.SIGNOUT, { method: 'POST', credentials: 'include'});
    if (rc = ((await res.status) == 200))  sou(); else sou();
    return rc;
};
export async function GetUserName(setA, setN) {
    let rc = false;
    let res = await fetch(API.GETUSERNAME, { credentials: 'include' });
    if (rc = ((await res.status) == 200)) setA((await res.json()).name); else  setN();
    return rc;
};
export function getSAID() {
    sessionStorage.SAID ? sessionStorage.SAID : (sessionStorage.SAID = (new Date()).getTime());
};
```

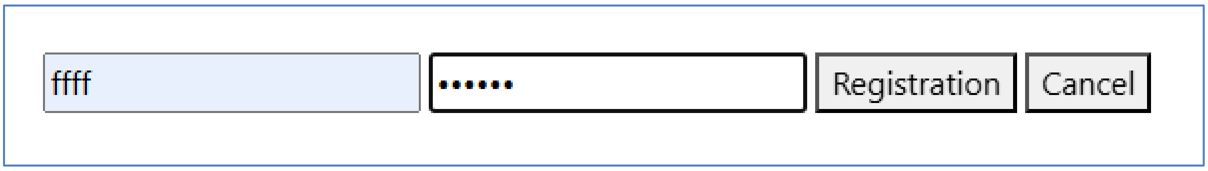
## 5. HTML-страница

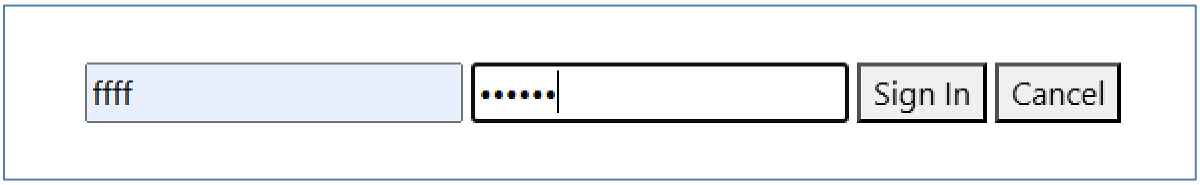
```
<h1>Module/Celebrities</h1>
<div class="row">
    <span class="col-2">
        <a href="/"              style="margin-left: 10px;"> Home </a>
        <a href="/Module/Jndex" style="margin-left: 10px;"> Jndex </a>
    </span>
    <span class="col-4"></span>
    <div class="col-5 d-flex justify-content-end ">
        <div id="STATE_N" style="margin-right:2%; display:none;">
            <button id="NSignIn" type="button"> Sign In</button>
            <button id="NRegistration" type="button">Registation</button>
        </div>
        <div id="STATE_A" style="margin-right: 2%; display:none;">
            <input type="text" id="AName" readonly />
            <button id="ASignOut" type="button">Sign Out</button>
        </div>
        <div id="STATE_R" style="margin-right: 2%; display:none;">
            <input id="RName" name="RName" type="text" placeholder="Login" />
            <input id="RPass" name="RPass" type="password" placeholder="Password" />
            <button id="RRegistration" type="button">Registration</button>
            <button id="RCancel" type="button">Cancel</button>
        </div>
        <div id="STATE_S" style="margin-right: 2%; display:none;">
            <input id="SName" name="NName" type="text" placeholder="Login" />
            <input id="SPass" name="NPass" type="password" placeholder="Password" />
            <button id="SSignIn" type="button">Sign In</button>
            <button id="SCancel" type="button">Cancel</button>
        </div>
    </div>
</div>
```

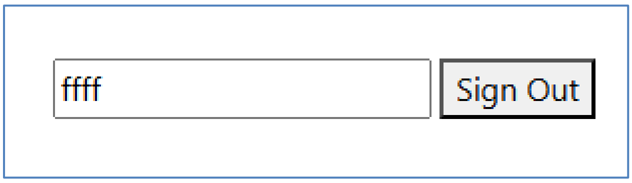
```
<script type="module">
    import  API, { SAID, SA, GetUserName, Registration, SignIn, SignOut } from '/js/LoginTagHelper.js';
    API.REGISTRATION = 'http://localhost:5000/api/Sec/Registration';
    API.SIGNIN       = 'http://localhost:5000/api/Sec/SignIn';
    API.SIGNOUT      = 'http://localhost:5000/api/Sec/SignOut';
    API.GETUSERNAME  = 'http://localhost:5000/api/Sec/GetUserName';

    const ButtonDissabled  = (flag) => {
            NSignIn.disabled = NRegistration.disabled = flag;
            RRegistration.disabled = SSignIn.disabled = flag;
            ASignOut.disabled = flag;
            RCancel.disabled = SCancel.disabled = flag;
    };

    let Visio = (state, name) => {
        if (state == SA.STATES.A) {
            AName.value = name||'';
            STATE_N.style.display = STATE_R.style.display = STATE_S.style.display = 'none';
            STATE_A.style.display = 'block';
        } else if (state == SA.STATES.N) {
            STATE_A.style.display = STATE_R.style.display = STATE_S.style.display = 'none';
            STATE_N.style.display = 'block';
        } else if (state == SA.STATES.R) {
            RName.value = RPass.value = '';
            STATE_A.style.display = STATE_N.style.display = STATE_S.style.display = 'none';
            STATE_R.style.display = 'block';
        } else if (state == SA.STATES.S) {
            SPass.value = '';
            SName.value = name||'';
            STATE_A.style.display = STATE_N.style.display = STATE_R.style.display = 'none';
            STATE_S.style.display = 'block';
        }
    };
```

```
    let said = new SAID(() => ButtonDissabled(true));
    let sa   = new SA(SA.START.A);
    GetUserName(name => {said.postSAID(); Visio(sa.state,name);},
                ()   => {sa.RunCommand(SA.COMMAND.SOU); said.postSAID(); Visio(sa.state); });


    sa.on(SA.STATES.RV, (nextstates) => Registration(RName.value, RPass.value,
                                            () => {sa.RunCommand(SA.COMMAND.SIN); said.postSAID(); Visio(sa.state, RName.value);},
                                            () => {sa.RunCommand(SA.COMMAND.REJ); said.postSAID(); Visio(sa.state);})
    );
    sa.on(SA.STATES.SV, (nextstates) => SignIn(SName.value, SPass.value,
                                      () => {sa.RunCommand(SA.COMMAND.LIN); said.postSAID(); Visio(sa.state, SName.value);},
                                      () => {sa.RunCommand(SA.COMMAND.ADE); said.postSAID(); Visio(sa.state);})
    );
    NRegistration.onclick = () => {sa.RunCommand(SA.COMMAND.REG); said.postSAID();  Visio(sa.state);};
    NSignIn.onclick       = () => {sa.RunCommand(SA.COMMAND.SIN); said.postSAID();  Visio(sa.state);};
    SSignIn.onclick       = () => {sa.RunCommand(SA.COMMAND.SIN); said.postSAID();  Visio(sa.state);};
    SCancel.onclick       = () => {sa.RunCommand(SA.COMMAND.CAN); said.postSAID();  Visio(sa.state);};
    RRegistration.onclick = () => {sa.RunCommand(SA.COMMAND.REG); said.postSAID();  Visio(sa.state);};
    RCancel.onclick       = () => {sa.RunCommand(SA.COMMAND.CAN); said.postSAID();  Visio(sa.state);};
    ASignOut.onclick      = () => {SignOut(() => {sa.RunCommand(SA.COMMAND.SOU); said.postSAID(); Visio(sa.state);})};

    console.log(sa);
    window.addEventListener('beforeunload', () => said.close());
</script>
```

## 6. Реализация




7. **КОНЕЦ.**